MODULE 7

NumPy

Topics Covered

Lecture 1 — NumPy Overview
Lecture 2 — NumPy Important Formulas
Lecture 3 — Mathematical Operations in NumPy
Lecture 4 — Statistical Functions & Other Important Concepts

Lecture 1: NumPy Overview

NumPy (Numerical Python) is the foundational library for numerical computing in Python. It provides the ndarray — a fast, memory-efficient multi-dimensional array — along with a huge library of mathematical functions that operate on it.

Why NumPy over Python lists?

Python list: [1, 2, 3] stores pointers to Python objects — slow and memory-heavy.

NumPy array: stores raw numbers in contiguous memory — up to 50× faster for large datasets.

This speed is critical for ML/AI — every tensor in TensorFlow & PyTorch is built on NumPy concepts.

Installation & Import

```
pip install numpy

import numpy as np   # 'np' is the universal alias
print(np.__version__)
```

Creating Arrays

```
# From Python list
a = np.array([1, 2, 3, 4, 5])          # 1D array
b = np.array([[1,2,3],[4,5,6]])         # 2D array (matrix)
c = np.array([[[1,2],[3,4]],[[5,6],[7,8]]])  # 3D array

# Built-in creators
np.zeros((3, 4))       # 3×4 matrix of 0s
np.ones((2, 3))        # 2×3 matrix of 1s
np.eye(4)              # 4×4 identity matrix
np.full((2, 2), 7)     # 2×2 matrix filled with 7
np.arange(0, 10, 2)    # [0, 2, 4, 6, 8]  (start, stop, step)
np.linspace(0, 1, 5)   # [0, 0.25, 0.5, 0.75, 1.0] — 5 evenly spaced
np.random.rand(3, 3)   # 3×3 random floats in [0, 1)
np.random.randint(1, 10, (2, 4))  # random integers
```

Array Attributes

Attribute	Description | Example
a.shape	Dimensions as tuple → (3, 4) means 3 rows, 4 cols
a.ndim	Number of dimensions → 2 for a matrix
a.size	Total number of elements → 12 for (3,4)
a.dtype	Data type of elements → int64, float32, etc.
a.itemsize	Bytes per element → 8 for float64

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape)     # (2, 3)
print(a.ndim)      # 2
```

Attribute	Description | Example

```
print(a.size)      # 6
print(a.dtype)     # int64
```

Lecture 2: NumPy Important Formulas

Indexing & Slicing

```
a = np.array([10, 20, 30, 40, 50])
print(a[0])          # 10   — first element
print(a[-1])         # 50   — last element
print(a[1:4])        # [20, 30, 40]
print(a[::2])        # [10, 30, 50]  — every 2nd

m = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(m[1, 2])       # 6    — row 1, col 2
print(m[0, :])       # [1, 2, 3]  — entire first row
print(m[:, 1])       # [2, 5, 8]  — entire second column
print(m[0:2, 1:3])   # [[2,3],[5,6]]  — submatrix
```

Reshaping & Transposing

```
a = np.arange(12)           # [0..11]
b = a.reshape(3, 4)         # 3×4 matrix  (must keep same size)
c = a.reshape(2, 2, 3)      # 3D: 2×2×3
d = a.reshape(-1, 3)        # NumPy infers rows automatically

m = np.array([[1,2,3],[4,5,6]])
print(m.T)                  # Transpose: (2,3) → (3,2)
print(m.flatten())          # Always returns 1D copy: [1,2,3,4,5,6]
print(m.ravel())            # 1D view (no copy if possible)
```

Stacking & Splitting

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.vstack([a, b])      # vertical:   [[1,2,3],[4,5,6]]
np.hstack([a, b])      # horizontal: [1,2,3,4,5,6]
np.concatenate([a, b]) # same as hstack for 1D

m = np.arange(16).reshape(4, 4)
np.vsplit(m, 2)        # split into 2 equal row-halves
np.hsplit(m, 2)        # split into 2 equal col-halves
```

Boolean & Fancy Indexing

```
a = np.array([10, 25, 3, 47, 8, 33])

# Boolean mask
mask = a > 20
```

```
print(mask)         # [False  True False  True False  True]
print(a[mask])      # [25, 47, 33]  — filter in one line
print(a[a % 2 == 0]) # [10, 8]  — even numbers only

# Fancy indexing — pick by index list
print(a[[0, 2, 5]])  # [10, 3, 33]
```

Broadcasting Rules

Broadcasting allows NumPy to perform operations on arrays of different shapes without copying data.

Rule	Description
Rule 1	If arrays differ in ndim, prepend 1s to the smaller shape
Rule 2	Dimensions of size 1 are stretched to match the other
Rule 3	If sizes don't match and neither is 1 → ValueError

```
a = np.array([[1],[2],[3]])   # shape (3,1)
b = np.array([10, 20, 30])   # shape (3,) → treated as (1,3)
print(a + b)
# [[11, 21, 31],
#  [12, 22, 32],
#  [13, 23, 33]]
```

Lecture 3: Mathematical Operations in NumPy

Element-wise Arithmetic

All standard operators (+, -, *, /, **, //, %) apply element-wise on arrays of the same shape:

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print(a + b)    # [11, 22, 33, 44]
print(b - a)    # [9, 18, 27, 36]
print(a * b)    # [10, 40, 90, 160]
print(b / a)    # [10., 10., 10., 10.]
print(a ** 2)   # [1, 4, 9, 16]

# Scalar operations (broadcasting with scalar)
print(a * 5)    # [5, 10, 15, 20]
print(a + 100)  # [101, 102, 103, 104]
```

Universal Functions (ufuncs)

ufuncs are optimised C-level functions that operate element-wise on arrays — much faster than Python loops:

ufunc	Description
np.sqrt(a)	Square root of each element
np.abs(a)	Absolute value
np.exp(a)	e^x for each element
np.log(a)	Natural log (ln)

ufunc	Description
np.log2(a)	Log base 2
np.log10(a)	Log base 10
np.sin/cos/tan(a)	Trigonometric functions
np.floor/ceil(a)	Round down / round up
np.round(a, n)	Round to n decimal places

```
a = np.array([1, 4, 9, 16, 25])
print(np.sqrt(a))       # [1., 2., 3., 4., 5.]
print(np.log(np.e))     # 1.0
print(np.exp([0,1,2]))  # [1., 2.718, 7.389]

angles = np.array([0, np.pi/2, np.pi])
print(np.sin(angles))   # [0., 1., 0.] (approx)
```

Matrix Operations

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

# Element-wise multiply (NOT matrix multiply)
print(A * B)             # [[5,12],[21,32]]

# Matrix multiplication
print(A @ B)             # [[19,22],[43,50]]
print(np.dot(A, B))      # same as @
print(np.matmul(A, B))   # same as @

# Matrix inverse & determinant
print(np.linalg.inv(A))  # inverse of A
print(np.linalg.det(A))  # determinant: -2.0

# Eigenvalues & eigenvectors
vals, vecs = np.linalg.eig(A)
print(vals)   # [-0.372, 5.372]
```

@ vs *
A @ B = matrix multiplication (dot product of rows × columns). A * B = element-wise multiplication (Hadamard product). In ML, weight × input uses @.

Lecture 4: Statistical Functions & Other Important Concepts

Aggregation & Statistical Functions

Function	Description
np.sum(a)	Sum of all elements
np.min(a) / np.max(a)	Minimum / Maximum value
np.argmin(a) / np.argmax(a)	Index of min / max

Function	Description
np.mean(a)	Arithmetic mean
np.median(a)	Median value
np.std(a)	Standard deviation
np.var(a)	Variance
np.cumsum(a)	Cumulative sum
np.percentile(a,q)	q-th percentile
np.unique(a)	Unique elements (sorted)

```
a = np.array([[1,2,3],[4,5,6],[7,8,9]])

print(np.sum(a))         # 45  — all elements
print(np.sum(a, axis=0)) # [12,15,18]  — column sums
print(np.sum(a, axis=1)) # [6, 15, 24] — row sums

print(np.mean(a))        # 5.0
print(np.std(a))         # 2.581...
print(np.median(a))      # 5.0
print(np.argmax(a))      # 8  (index in flattened array)
print(np.percentile(a, 75))  # 6.5
```

axis parameter
axis=0 → operate down the rows (column-wise result)
axis=1 → operate across columns (row-wise result)
No axis → operate on entire flattened array

Sorting & Searching

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])

print(np.sort(a))          # [1,1,2,3,4,5,6,9] — returns copy
print(np.argsort(a))       # indices that would sort: [1,3,6,0,2,4,7,5]

print(np.where(a > 4))     # (array([4, 5, 7]),) — indices where True
print(np.where(a > 4, 1, 0))  # replace: 1 if >4, else 0

print(np.searchsorted([1,3,5,7], 4))  # 2 — insertion point
```

Random Module

```
np.random.seed(42)              # fix seed for reproducibility

np.random.rand(3, 3)            # uniform [0, 1)
np.random.randn(3, 3)           # standard normal (mean=0, std=1)
np.random.randint(1, 100, 10)   # 10 random ints in [1, 100)
np.random.choice([1,2,3,4,5], 3, replace=False)  # sample without replacement
np.random.shuffle(a)            # shuffle in-place

# Normal distribution with custom mean & std
np.random.normal(loc=50, scale=10, size=1000)
```

Data Type Control

```
a = np.array([1, 2, 3], dtype=np.float32)   # specify dtype
b = a.astype(np.int64)                       # cast dtype

# Common dtypes:
# np.int8, int16, int32, int64
# np.float16, float32, float64
# np.bool_, np.complex64

# float32 vs float64 in ML:
# float32 = 4 bytes, faster on GPU
# float64 = 8 bytes, more precise (default)
```

Copying: View vs Copy

```
a = np.array([1, 2, 3, 4, 5])

b = a          # NO copy — b is just another name for a
b[0] = 99
print(a)       # [99, 2, 3, 4, 5] — a changed too!

c = a.copy()   # TRUE copy — independent
c[0] = 0
print(a)       # [99, 2, 3, 4, 5] — a unchanged

# Views: slices share memory
v = a[1:4]     # view — modifying v modifies a
v[0] = 500
print(a)       # [99, 500, 3, 4, 5]
```

View vs Copy trap
Slices in NumPy return VIEWS (shared memory), not copies.
Always use .copy() when you need an independent array to avoid silent bugs.

End of Module 7 Notes — Generative AI Course